Reporte: Markdown, Git, GitHub y Hugo

Paradigmas de la programacion Practica 0

Markdown · Git · GitHub · Hugo · GitHub Actions

Materia: Paradigmas de la Programacion **Nombre:** *Leonel Cajeme Garcia*
Matricula: *379154*
Grupo: *Ing. Software 941*
Profesor: *Jose Carlos Gallegos Mariscal*
Fecha: *20 de febrero de 2026*

> 🗀 Repositorio en GitHub: github.com/tu-usuario/tu-repositorio
> 🌐 Página estática (GitHub Pages): tu-usuario.github.io/tu-repositorio

---

Tabla de Contenidos

- Reporte: Markdown, Git, GitHub y Hugo
- Paradigmas de la programacion Practica 0
  - Markdown · Git · GitHub · Hugo · GitHub Actions
  - Tabla de Contenidos
  - Introducción
  - Sesión 1 – Markdown
    - ¿Qué es Markdown?
    - ¿Para qué se utiliza?
    - Sintaxis de Markdown
      - Encabezados
      - Énfasis de texto
      - Listas
      - Vínculos e imágenes
      - Bloques de código
      - Tablas
      - Citas
      - Línea horizontal
      - Checkboxes (listas de tareas)
  - Sesión 2 – Git y GitHub
    - ¿Qué es Git?
    - ¿Qué es GitHub?
    - Comandos esenciales de Git
      - Configuración inicial
      - Inicializar y clonar repositorios

        - Ciclo básico de trabajo
        - Ramas (branches)
        - Trabajar con repositorios remotos
        - Otros comandos útiles
    - Crear un repositorio y subir información
  - Sesión 3 – Hugo y GitHub Actions
    - ¿Qué es Hugo?
    - ¿Qué es GitHub Actions?
    - Crear un sitio estático con Hugo
    - Subir el sitio a GitHub
    - Configurar GitHub Actions para publicar el sitio en GitHub Pages
  - Mi Portafolio
  - Recursos útiles
  - Conclusiones

Introducción

En el desarrollo de software moderno, dominar herramientas de documentación, control de versiones y despliegue es fundamental. Este reporte cubre tres sesiones prácticas donde se aprendió a usar **Markdown** para redactar documentos con formato, **Git y GitHub** para el control de versiones y trabajo colaborativo, y finalmente **Hugo con GitHub Actions** para generar y publicar páginas estáticas de manera automatizada.

> **Nota importante:** Aunque trabajemos en Windows (CMD o PowerShell), se recomienda utilizar **Git Bash**, ya que soporta comandos de Linux. En entornos profesionales y en la nube (AWS, GCP, Azure, etc.) el sistema operativo predominante es Linux o Unix (macOS), por lo que acostumbrarse a la línea de comandos desde ahora es una ventaja clave.

Sesión 1 – Markdown

¿Qué es Markdown?

Markdown es un lenguaje de marcado ligero creado en 2004 por **John Gruber** con la colaboración de Aaron Swartz. Su propósito es permitir escribir texto con formato de una manera sencilla y legible en texto plano, que luego puede convertirse fácilmente a HTML u otros formatos.

A diferencia de HTML, Markdown es legible incluso sin ser procesado, lo que lo hace ideal para documentación técnica, archivos `README`, blogs y notas.

¿Para qué se utiliza?

Markdown se usa ampliamente en:

- **GitHub / GitLab**: archivos `README.md`, wikis y comentarios en issues/pull requests.
- **Blogs y sitios estáticos**: plataformas como Hugo, Jekyll o Ghost usan Markdown para el contenido.
- **Documentación técnica**: proyectos de software que documentan su API o guías de uso.
- **Herramientas de notas**: Notion, Obsidian, VS Code y otros editores lo soportan nativamente.
- **Correos y mensajería**: Slack, Discord y Teams soportan subconjuntos de Markdown.

Sintaxis de Markdown

A continuación se muestra la sintaxis esencial de Markdown:

Encabezados

```
# Encabezado H1
## Encabezado H2
### Encabezado H3
#### Encabezado H4
```

Énfasis de texto

```
**Texto en negrita**
*Texto en cursiva*
~~Texto tachado~~
`código en línea`
```

Resultado:

- **Texto en negrita**
- *Texto en cursiva*
- ~~Texto tachado~~
- `código en línea`

Listas

```
- Elemento sin orden
- Otro elemento
  - Sub-elemento

1. Primer elemento
2. Segundo elemento
3. Tercer elemento
```

Vínculos e imágenes

```
[Texto del enlace](https://www.ejemplo.com)
![Texto alternativo](ruta/imagen.png)
```

Bloques de código

```
```python
def hola_mundo():
    print("Hola, Mundo!")
```
```

Tablas

```
| Columna 1 | Columna 2 | Columna 3 |
|-----------|-----------|-----------|
| Dato 1    | Dato 2    | Dato 3    |
| Dato 4    | Dato 5    | Dato 6    |
```

Resultado:

Columna 1	Columna 2	Columna 3
Dato 1	Dato 2	Dato 3
Dato 4	Dato 5	Dato 6

Citas

```
> Esto es una cita.
> Puede abarcar varias líneas.
```

Línea horizontal

```
---
```

Checkboxes (listas de tareas)

```
- [x] Tarea completada
- [ ] Tarea pendiente
```

Sesión 2 – Git y GitHub

¿Qué es Git?

Git es un sistema de **control de versiones distribuido** creado por **Linus Torvalds** en 2005 para gestionar el desarrollo del kernel de Linux. Permite rastrear cambios en archivos a lo largo del tiempo, trabajar en paralelo

con múltiples desarrolladores y revertir cambios cuando sea necesario.

Sus principales características son:

- **Distribuido**: cada desarrollador tiene una copia completa del historial del repositorio.
- **Velocidad**: operaciones locales sin necesidad de conexión constante a un servidor.
- **Ramificación (branching)**: permite crear ramas de trabajo independientes y fusionarlas.
- **Integridad**: usa hashes SHA-1 para garantizar la integridad de los datos.

¿Qué es GitHub?

GitHub es una plataforma de alojamiento en la nube para repositorios Git, adquirida por Microsoft en 2018. No solo almacena el código fuente, sino que ofrece herramientas de colaboración como:

- **Pull Requests**: proponer y revisar cambios antes de integrarlos.
- **Issues**: gestión de tareas, errores y mejoras.
- **GitHub Actions**: automatización de flujos de trabajo (CI/CD).
- **GitHub Pages**: hospedaje gratuito de páginas web estáticas.
- **Wikis y Proyectos**: documentación y gestión de proyectos.

Comandos esenciales de Git

Configuración inicial

```
# Configurar nombre de usuario y correo (obligatorio la primera vez)
git config --global user.name "Tu Nombre"
git config --global user.email "tucorreo@ejemplo.com"

# Ver la configuración actual
git config --list
```

Inicializar y clonar repositorios

```
# Inicializar un repositorio nuevo en la carpeta actual
git init

# Clonar un repositorio existente
git clone https://github.com/usuario/repositorio.git
```

Ciclo básico de trabajo

```
# Ver el estado de los archivos
git status

# Agregar archivos al área de staging
git add archivo.txt        # un archivo específico
```

```
git add .                   # todos los cambios

# Crear un commit (guardar los cambios con mensaje)
git commit -m "Descripción del cambio"

# Ver el historial de commits
git log
git log --oneline           # versión compacta
```

Ramas (branches)

```
# Ver ramas existentes
git branch

# Crear una nueva rama
git branch nombre-rama

# Cambiar a otra rama
git checkout nombre-rama

# Crear y cambiar a una nueva rama en un solo paso
git checkout -b nueva-rama

# Fusionar una rama con la rama actual
git merge nombre-rama
```

Trabajar con repositorios remotos

```
# Ver los remotos configurados
git remote -v

# Agregar un remoto
git remote add origin https://github.com/usuario/repositorio.git

# Subir cambios al remoto
git push origin main

# Descargar y fusionar cambios del remoto
git pull origin main

# Descargar cambios sin fusionar
git fetch origin
```

Otros comandos útiles

```
# Deshacer cambios en un archivo (antes del staging)
git checkout -- archivo.txt

# Quitar un archivo del staging
git reset HEAD archivo.txt

# Ver diferencias entre el área de trabajo y el staging
git diff

# Etiquetar una versión
git tag v1.0.0
```

Crear un repositorio y subir información

Los pasos para crear un repositorio en GitHub y subir código son los siguientes:

1. Crear el repositorio en GitHub

- Ir a github.com e iniciar sesión.
- Hacer clic en el botón **"New"** o **"+"** → **New repository**.
- Asignar un nombre, descripción opcional y elegir si será público o privado.
- (Opcional) Inicializar con README, `.gitignore` y licencia.
- Hacer clic en **"Create repository"**.

2. Conectar el repositorio local con GitHub

```
# Inicializar el repositorio local
git init

# Agregar todos los archivos
git add .

# Primer commit
git commit -m "Primer commit: inicio del proyecto"

# Conectar con el repositorio remoto en GitHub
git remote add origin https://github.com/tu-usuario/tu-repositorio.git

# Cambiar la rama principal a 'main' si es necesario
git branch -M main

# Subir al repositorio remoto
git push -u origin main
```

3. Configurar SSH (recomendado)

Para no ingresar usuario y contraseña en cada push, se puede configurar una llave SSH:

```
# Generar llave SSH
ssh-keygen -t ed25519 -C "tucorreo@ejemplo.com"

# Copiar la llave pública
cat ~/.ssh/id_ed25519.pub

# Luego agregar esta llave en GitHub: Settings → SSH and GPG Keys → New SSH key
```

Sesión 3 – Hugo y GitHub Actions

¿Qué es Hugo?

Hugo es un generador de sitios estáticos de código abierto escrito en **Go**, reconocido por ser uno de los más rápidos del mercado. Convierte archivos Markdown y plantillas en un sitio web estático listo para publicar, sin necesidad de base de datos ni servidor backend.

Ventajas de Hugo:

- **Velocidad extrema**: capaz de generar miles de páginas en milisegundos.
- **Portabilidad**: el sitio generado es solo HTML, CSS y JS — desplegable en cualquier servidor.
- **Temas**: amplia biblioteca de temas gratuitos en themes.gohugo.io.
- **Markdown nativo**: el contenido se escribe en Markdown.
- **Sin dependencias en producción**: no requiere PHP, Ruby, Node.js, etc.

¿Qué es GitHub Actions?

GitHub Actions es la plataforma de **integración y entrega continua (CI/CD)** de GitHub. Permite automatizar flujos de trabajo directamente desde el repositorio, como compilar el sitio de Hugo y desplegarlo en GitHub Pages cada vez que se hace un `push`.

Conceptos clave:

- **Workflow**: archivo YAML que define el flujo de trabajo (`.github/workflows/`).
- **Job**: conjunto de pasos que se ejecutan en una máquina virtual.
- **Step**: una tarea individual dentro de un job (puede ejecutar comandos o usar acciones).
- **Action**: módulo reutilizable del marketplace de GitHub Actions.
- **Trigger**: evento que dispara el workflow (ej. `push` a la rama `main`).

Crear un sitio estático con Hugo

1. Instalar Hugo

```
# En macOS (con Homebrew)
brew install hugo

# En Windows (con Chocolatey)
choco install hugo-extended
```

```
# En Linux (Debian/Ubuntu)
sudo apt install hugo

# Verificar instalación
hugo version
```

2. Crear un nuevo sitio

```
# Crear el sitio
hugo new site mi-portafolio
cd mi-portafolio

# Inicializar repositorio Git
git init
```

3. Agregar un tema

```
# Agregar tema como submódulo de Git (ejemplo: tema Ananke)
git submodule add https://github.com/theNewDynamic/gohugo-theme-ananke.git
themes/ananke

# Configurar el tema en hugo.toml
echo 'theme = "ananke"' >> hugo.toml
```

4. Crear contenido

```
# Crear una nueva entrada de blog
hugo new posts/mi-primer-post.md
```

Esto crea un archivo en `content/posts/mi-primer-post.md` con frontmatter:

```
---
title: "Mi Primer Post"
date: 2024-01-01T00:00:00-06:00
draft: true
---

Aquí va el contenido del post en **Markdown**.
```

5. Vista previa local

```
# Iniciar el servidor de desarrollo (incluye borradores con -D)
hugo server -D
```

```
# Acceder en el navegador: http://localhost:1313
```

6. Generar el sitio estático

```
# Compilar el sitio (genera la carpeta /public)
hugo
```

Subir el sitio a GitHub

```
# Agregar todos los archivos al staging
git add .

# Commit inicial
git commit -m "Agregar sitio Hugo"

# Conectar con el repositorio en GitHub
git remote add origin https://github.com/tu-usuario/mi-portafolio.git
git branch -M main

# Subir al repositorio
git push -u origin main
```

Configurar GitHub Actions para publicar el sitio en GitHub Pages

1. Habilitar GitHub Pages

- Ir al repositorio en GitHub.
- **Settings** → **Pages**.
- En **Source**, seleccionar **GitHub Actions**.

2. Crear el archivo de workflow

Crear el archivo `.github/workflows/hugo.yml`:

```
name: Deploy Hugo site to GitHub Pages

on:
  push:
    branches:
      - main
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write
```

```yaml
concurrency:
  group: "pages"
  cancel-in-progress: false

defaults:
  run:
    shell: bash

jobs:
  build:
    runs-on: ubuntu-latest
    env:
      HUGO_VERSION: 0.128.0
    steps:
      - name: Install Hugo CLI
        run: |
          wget -O ${{ runner.temp }}/hugo.deb \

https://github.com/gohugoio/hugo/releases/download/v${HUGO_VERSION}/hugo_extended_
${HUGO_VERSION}_linux-amd64.deb \
          && sudo dpkg -i ${{ runner.temp }}/hugo.deb

      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          submodules: recursive
          fetch-depth: 0

      - name: Setup Pages
        id: pages
        uses: actions/configure-pages@v5

      - name: Build with Hugo
        env:
          HUGO_CACHEDIR: ${{ runner.temp }}/hugo_cache
          HUGO_ENVIRONMENT: production
        run: |
          hugo \
            --gc \
            --minify \
            --baseURL "${{ steps.pages.outputs.base_url }}/"

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: ./public

  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    needs: build
```

```
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

3. Hacer push del workflow

```
git add .github/
git commit -m "Agregar GitHub Actions workflow para Hugo"
git push origin main
```

Después del push, GitHub Actions compilará automáticamente el sitio y lo publicará en `https://tu-usuario.github.io/mi-portafolio/`.

Mi Portafolio

Recurso	Enlace
🗀 Repositorio en GitHub	github.com/tu-usuario/tu-repositorio
🌐 Página estática en GitHub Pages	tu-usuario.github.io/tu-repositorio

> ⚠ *Sustituye los enlaces anteriores con los URLs reales de tu portafolio.*

Recursos útiles

A continuación se listan los recursos recomendados para profundizar en los temas de este reporte:

Tema	Recurso
Markdown para principiantes	Fazt Code – Markdown, Curso Práctico
Línea de comandos en Windows	Introducción a la línea de comando en Windows
Directorios y archivos con Git Bash	Cómo crear directorios y archivos con Git Bash
Comandos básicos de Linux	Introducción a la línea de comandos Linux
Git y GitHub desde cero	Fazt Code – Git and Github | Practical Course
Cuenta en GitLab	gitlab.com/users/sign_in
SSH Key en GitLab	Cómo configurar SSH Key en GitLab
SSH en Windows con PowerShell	Generar llave SSH en Windows
Quickstart Hugo	gohugo.io/getting-started/quick-start
Quickstart GitLab + Hugo	docs.gitlab.com/ee/tutorials/hugo

Conclusiones

A lo largo de estas tres sesiones se adquirieron habilidades fundamentales para cualquier desarrollador de software moderno:

- **Markdown** permite documentar proyectos de manera profesional usando solo texto plano, con una curva de aprendizaje mínima y compatibilidad universal.
- **Git** proporciona control total sobre el historial de un proyecto, permitiendo experimentar sin miedo a perder trabajo y colaborar eficientemente en equipo.
- **GitHub** extiende las capacidades de Git con herramientas de colaboración, visibilidad pública del trabajo y servicios adicionales como Pages y Actions.
- **Hugo** demuestra que es posible tener un sitio web rápido, seguro y fácil de mantener sin necesidad de frameworks complejos ni bases de datos.
- **GitHub Actions** cierra el ciclo automatizando el despliegue: con un simple `git push`, el sitio se compila y publica automáticamente.

La combinación de estas herramientas forma la base de un **portafolio profesional** versionado, publicado y mantenido de manera eficiente.